

HDOC Day-18

Menu-driven C Program using switch-case and void user-defined functions

Problem Statement

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and a simple function call. Each function reads the required input, processes it, and displays the result.

- 1. Check whether it is a Strong number.
- 2. Check whether it is an Armstrong number.
- 3. Check whether it is a Harshad number.
- 4. Check whether it is a Duck number.

Algorithm

Main Program

1. Start.
2. Display four menu choices.
3. Read the user choice.
4. Use switch-case to call the corresponding void function: strongNumber(), armstrongNumber(), harshadNumber(), or duckNumber().
5. For an invalid choice, display an error message.
6. Stop.

Function Logic

strongNumber(): Read n. For every digit, calculate digit factorial and add it. If the sum equals n, it is a Strong number.

armstrongNumber(): Read n. Count its digits. Add each digit raised to that digit-count. If the sum equals n, it is an Armstrong number.

harshadNumber(): Read n. Find the sum of its digits. If n is divisible by the digit sum, it is a Harshad number.

duckNumber(): Read n. Check whether any digit is 0. Since the input is a positive integer, a zero occurring within its decimal digits makes it a Duck number.

Test Case Table

Choice	Operation	Input	Expected Output	Reason / Check
1	Strong	145	Strong number	$1! + 4! + 5! = 145$
1	Strong	123	Not a Strong number	Factorial sum is not 123
2	Armstrong	153	Armstrong number	$1^3 + 5^3 + 3^3 = 153$
2	Armstrong	123	Not an Armstrong number	Cube sum is not 123
3	Harshad	18	Harshad number	18 is divisible by $1+8=9$
3	Harshad	19	Not a Harshad number	19 is not divisible by 10
4	Duck	1023	Duck number	Contains digit 0
4	Duck	1234	Not a Duck number	Contains no digit 0

C Program

```
#include <stdio.h>
```

```
void strongNumber();
void armstrongNumber();
void harshadNumber();
void duckNumber();
```

```
int main()
{
    int choice;

    printf("1. Check Strong number\n");
    printf("2. Check Armstrong number\n");
    printf("3. Check Harshad number\n");
    printf("4. Check Duck number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
```

```
    switch(choice)
    {
        case 1: strongNumber(); break;
        case 2: armstrongNumber(); break;
        case 3: harshadNumber(); break;
        case 4: duckNumber(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}
```

```
void strongNumber()
{
    int n, temp, digit, sum = 0, i, fact;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        fact = 1;
        for(i = 1; i <= digit; i++)
            fact = fact * i;
        sum = sum + fact;
        temp = temp / 10;
    }
    if(sum == n)
        printf("%d is a Strong number.\n", n);
    else
        printf("%d is not a Strong number.\n", n);
}
```

```
void armstrongNumber()
{
    int n, temp, digit, digits = 0, sum = 0, i, power;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0) { digits++; temp = temp / 10; }
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        power = 1;
        for(i = 1; i <= digits; i++) power = power * digit;
        sum = sum + power;
    }
}
```

```

        temp = temp / 10;
    }
    if(sum == n)
        printf("%d is an Armstrong number.\n", n);
    else
        printf("%d is not an Armstrong number.\n", n);
}

```

```

void harshadNumber()
{
    int n, temp, digit, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        sum = sum + digit;
        temp = temp / 10;
    }
    if(n % sum == 0)
        printf("%d is a Harshad number.\n", n);
    else
        printf("%d is not a Harshad number.\n", n);
}

```

```

void duckNumber()
{
    int n, temp, digit, hasZero = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        if(digit == 0) { hasZero = 1; break; }
        temp = temp / 10;
    }
    if(hasZero)
        printf("%d is a Duck number.\n", n);
    else
        printf("%d is not a Duck number.\n", n);
}

```

Sample Executions

Choice 1 - Strong Number

Choice: 1

Input: 145

Output: 145 is a Strong number.

Choice 2 - Armstrong Number

Choice: 2

Input: 153

Output: 153 is an Armstrong number.

Choice 3 - Harshad Number

Choice: 3

Input: 18

Output: 18 is a Harshad number.

Choice 4 - Duck Number

Choice: 4

Input: 1023

Output: 1023 is a Duck number.

Screenshots

Paste your actual terminal/compiler screenshots in the spaces below.

Screenshot 1 - Source Code / Compilation

```
#include <stdio.h>
void strongNumber();
void armstrongNumber();
void harshadNumber();
void duckNumber();
int main()
{
    int choice;
    printf("1. Check Strong number\n");
    printf("2. Check Armstrong number\n");
    printf("3. Check Harshad number\n");
    printf("4. Check Duck number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1: strongNumber(); break;
        case 2: armstrongNumber(); break;
        case 3: harshadNumber(); break;
        case 4: duckNumber(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}
void strongNumber()
{
    int n, temp, digit, sum = 0, i, fact;
```

```

scanf("%d", &n);
temp = n;
while(temp > 0)
{
    digit = temp % 10;
    fact = 1;
    for(i = 1; i <= digit; i++)
        fact = fact * i;
    sum = sum + fact;
    temp = temp / 10;
}
if(sum == n)
    printf("%d is a Strong number.\n", n);
else
    printf("%d is not a Strong number.\n", n);
}

void armstrongNumber()
{
    int n, temp, digit, digits = 0, sum = 0, i, power;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0) { digits++; temp = temp / 10; }
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;

```

```

        power = 1;
        for(i = 1; i <= digits; i++) power = power * digit;
        sum = sum + power;
        temp = temp / 10;
    }
    if(sum == n)
        printf("%d is an Armstrong number.\n", n);
    else
        printf("%d is not an Armstrong number.\n", n);
}

void harshadNumber()
{
    int n, temp, digit, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        sum = sum + digit;
        temp = temp / 10;
    }
    if(n % sum == 0)
        printf("%d is a Harshad number.\n", n);
    else
        printf("%d is not a Harshad number.\n", n);
}

```

```

}

void duckNumber()
{
    int n, temp, digit, hasZero = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;
    while(temp > 0)
    {
        digit = temp % 10;
        if(digit == 0) { hasZero = 1; break; }
        temp = temp / 10;
    }
    if(hasZero)
        printf("%d is a Duck number.\n", n);
    else
        printf("%d is not a Duck number.\n", n);
}

```

```
(kali㉿kali)-[~/Desktop]
$ vi sample14.c

(kali㉿kali)-[~/Desktop]
$ gcc sample14.c -o sample14 -lm
```

Screenshot 2 - Choice 1 Output (Strong Number)

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 1
Enter a positive integer: 123
123 is not a Strong number.
```

Screenshot 3 - Choice 2 Output (Armstrong Number)

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 2
Enter a positive integer: 123
123 is not an Armstrong number.
```

Screenshot 4 - Choice 3 Output (Harshad Number)

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 3
Enter a positive integer: 19
19 is not a Harshad number.
```

Screenshot 5 - Choice 4 Output (Duck Number)

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 4
Enter a positive integer: 1023
1023 is a Duck number.
```

```
(kali㉿kali)-[~/Desktop]
$ ./sample14
1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
Enter your choice: 4
Enter a positive integer: 1234
1234 is not a Duck number.
```